

Opdracht Kubernetes cluster

Lauwers Bjorn
R1060683
2WT-CCS

Inhoud

1. PROJECTOVERZICHT	3
2. SYSTEEMVEREISTEN	3
2.1. Hostmachine (Windows)	3
2.1.1. Benodigde tools	3
2.1.2. VM opstarten en gebruiken	3
2.2. Virtuele machine (ubuntu 22.04)	4
2.2.1. Benodigde tools	4
2.2.2. Installatiecommando's	4
3. PROJECTBESTANDEN	5
3.1. ./vagrantfile	5
3.2. ./github/workflows/build-image.yaml	5
3.3. ./lego	6
3.4. ./Webstack/api	6
3.5. ./Webstack/db	6
3.6. ./Webstack/frontend	6
3.7. ./Webstack/k8s/argocd	7
3.8. ./Webstack/k8s/base	7
3.9. ./Webstack/k8s/cert-manager	7
3.10. ./Webstack/k8s/monitoring	8
4. OPZETTEN KUBERNETES CLUSTER	8
4.1. Aanmaken van de cluster, ingress en namespaces	8
4.2. Integreer de app in de cluster	9
4.3. Monitoring opzetten	9
4.4. ArgoCD opzetten:	9
4.5. Certificaat toevoegen en tls opzetten	10
4.6. Eindcontrole	10
4.7. Troubleshooting	10
4.7.1. Ingress geeft 404	10
4.7.2. API kan DB niet bereiken	10
4.7.3. ArgoCD overschrijft manuele Kubectl edits	11

1. Projectoverzicht

Dit project bouwt een eenvoudige webstack met 3 lagen:

- Frontend: statische pagina via Lighttpd
- API: FastAPI (Python)
- Database: MariaDB

In Kubernetes wordt dit uitgebreid met:

- NGINX Ingress controller
- TLS-secrets op basis van een lego Let's Encrypt certificaat
- monitoring via kube-prometheus-stack + custom ServiceMonitor
- Grafana dashboard als ConfigMap
- ArgoCD Application voor GitOps sync

2. Systemvereisten

2.1. Hostmachine (Windows)

2.1.1. Benodigde tools

Tool	Functie
VirtualBox	De 'motor' die de hardware van je computer (zoals CPU en RAM) veilig deelt met een gast-besturingssysteem, zonder je eigen systeem aan te passen.
Vagrant	Automatiseert het opzetten van VirtualBox-machines via een simpel tekstbestand (de Vagrantfile).
VBguest	Voor functies zoals het delen van mappen en netwerkverbindingen tussen host en gast-machine.

2.1.2. VM opstarten en gebruiken

Ga naar de projectrootmap, open hier powershell en start de VM met het commando 'vagrant up'. Vagrant kijkt in de vagrantfile voor configuratie, systeembronnen, netwerkinstellingen, shared folders en provisioning.

Als de virtuele machine opgestart is, wordt het commando 'vagrant ssh' gebruikt om connectie te maken en te interageren met de virtuele machine.

Om de machine terug veilig af te sluiten, wordt het 'vagrant halt' commando gebruikt.

2.2. Virtuele machine (ubuntu 22.04)

2.2.1. Benodigde tools

Eenmaal geconnecteerd met de virtuele machine, kan deze voorbereid worden met volgende tools: Curl, Docker (+ Docker Compose), Kind, Kubectl en Helm.

Tool	Functie
Curl	Wordt gebruikt om installatiebestanden, scripts en API-data veilig te downloaden via het internet en is nodig om andere programma's te installeren.
Docker (+ Docker Compose)	Is een platform om applicaties te verpakken in lichtgewicht 'containers'. Docker Compose helpt bij het draaien van meerdere samenwerkende containers. Docker is nodig voor KinD (kubernetes in docker) want daar zit alles in 'containers'.
Kind	Een tool waarmee lokale Kubernetes-clusters draaien met behulp van Docker-containers als "nodes".
Kubectl	Hiermee geef je commando's aan je Kubernetes-cluster, zoals het starten van applicaties of het controleren van de status van je systeem.
Helm	Is een package manager voor Kubernetes en hiermee kan je Helm-charts downloaden. Dit zijn verzamelingen van bestanden die alle benodigde Kubernetes resources beschrijft. In plaats van losse YAML-bestanden handmatig te beheren, verpakt de chart de hele applicatie in één herbruikbaar pakket.

2.2.2. Installatiecommando's

Installatiecommando's
<code>sudo apt update</code>
<code>sudo apt install apt-transport-https curl</code>
<code>curl -fsSL https://download.docker.com/linux/ubuntu/gpg sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg</code>
<code>echo "deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \$(. /etc/os-release && echo "\$VERSION_CODENAME") stable" sudo tee /etc/apt/sources.list.d/docker.list > /dev/null</code>
<code>sudo apt update</code>
<code>sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin</code>
<code>[\$(uname -m) = x86_64] && curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.31.0/kind-linux-amd64</code>
<code>chmod +x ./kind</code>
<code>sudo mv ./kind /usr/local/bin/kind</code>
<code>curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/Kubectl"</code>
<code>sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl</code>
<code>curl https://raw.githubusercontent.com/Helm/Helm/main/scripts/get-Helm-3 bash</code>

3. Projectbestanden

In onderstaand hoofdstuk worden alle files beschreven en wordt gedetailleerd beschreven wat hun functie is. Alle files zijn te vinden op github (https://github.com/LauwersB/BL_webstack-project/tree/main).

3.1. ./vagrantfile

De ./vagrantfile definieert één Ubuntu-VM met VirtualBox, met vaste resources (8 GB RAM, 4 vCPU), een private IP en rsync-mapdeling met de projectmap.

Daarnaast bevat het twee lifecycle-triggers. Bij het afsluiten van de VM worden alle deployments binnen de namespaces monitoring en argocd teruggedeschaald naar 0 replica's om resources en kosten te besparen. Wanneer de VM vervolgens weer wordt opgestart, gebeurt dit proces stapsgewijs: na een initiële wachtperiode wordt eerst de monitoring opgeschaald naar 1 replica, waarna er opnieuw wordt gewacht voordat ook ArgoCD weer naar 1 replica wordt geschaald.

De shellblokken zijn defensief geschreven: als Kubectl niet beschikbaar is, stoppen ze zonder fout.

3.2. ./github/workflows/build-image.yaml

Deze CI/CD-flow bouwt bij elke push naar main automatisch images op Dockerhub en werkt daarna de Kubernetes manifests bij, zodat de versienummers aangepast worden naar de nieuwe images. Dit proces verloopt als volgt:

Trigger	De workflow start bij een push naar main.
Job build_api	Ze checkt de code uit, bouwt de API-image vanuit de API-map en voorziet deze van een tag met de commit SHA. Daarna logt ze in op Docker Hub en pusht ze de image.
Job build_frontend	Ze wacht op build_api, bouwt daarna de frontend image uit frontend, tagt ook met dezelfde commit SHA en pusht die.
Manifest update	In dezelfde frontend-job past ze image-tags aan in api.yaml en frontend.yaml, zodat Kubernetes de recent gebouwde SHA-images exact gebruikt.
Commit alleen bij wijziging	Als er iets veranderde in de manifests, commit en pusht ze terug naar main; anders slaat ze die stap over.
Bot-loop bescherming	Beide jobs draaien alleen als actor niet github-actions[bot] is, zodat de commit van de workflow zelf geen oneindige loop veroorzaakt.

Deze opzet heeft verschillende voordelen.

Reproduceerbaarheid	Taggen met commit SHA maakt elke image exact herleidbaar naar de broncode.
Consistentie	API en frontend krijgen dezelfde versie-identiteit (dezelfde SHA), wat deploys voorspelbaarder maakt.
Werken zoals GitOps	Door manifests in Git te updaten, blijft de gewenste clusterstate in de repo zichtbaar en te beheren per versie.
Veilige automatisatie	Permissions contents: write is nodig om terug te committen, daarnaast voorkomt de actor-check runaway pipelines.
Operationele robuustheid	De workflow commit niet "blind", enkel als sed effectief wijzigingen maakte.

3.3. ./lego

Deze map staat niet op Github. Deze map bevat mijn publiek certificaat van let's encrypt. Dit certificaat is aangevraagd met mijn domeinnaam op DuckDNS met behulp van volgend commando:

```
docker run -v ./lego:/opt/lego -e DUCKDNS_TOKEN=xxxxxxx-xxxx-xxxx-xxxxxxxxxxx goacme/lego:latest --email=<your@email.be> --dns duckdns --accept-tos --path=/opt/lego -d yourdomain.duckdns.org run
```

3.4. ./Webstack/api

Deze map is de backend van de applicatie. Ze vormt de laag tussen frontend en database: ze leest en wijzigt data in MariaDB, stelt HTTP-endpoints beschikbaar voor de frontend en exposeert extra endpoints voor health checks en monitoring. De map bevat dus niet alleen de Python-app zelf, maar ook alles wat nodig is om die app als container te bouwen en consistent te laten draaien.

- [main.py](#): Dit is de FastAPI-app. Hier worden de CORS-instellingen toegepast, de routes aangemaakt en de databaseverbinding gebruikt. Functioneel doet dit bestand vier dingen: het leest een gebruiker op via /user, het wijzigt die gebruiker via /user/update, het biedt /health aan voor Kubernetes health probes, en het biedt /metrics aan zodat Prometheus statistieken kan ophalen. Dit is op deze manier opgezet omdat de API niet alleen businesslogica moet aanbieden, maar ook operationeel bruikbaar moet zijn in een cluster.
- [config.py](#): Dit bestand centraliseert configuratie. Het leest databasehost, databasenaam, gebruikersnaam, wachtwoord en CORS-origins uit omgevingsvariabelen. Zo blijft de code zelf omgevingsonafhankelijk, want dezelfde applicatie kan draaien in een lokaal Docker Compose, in CI en in Kubernetes, zonder harde waarden in de broncode.
- [Dockerfile](#): Dit bouwt de containerimage van de API. Het neemt een lichte Python-basisimage, installeert de dependencies, kopieert de applicatiecode en start daarna Uvicorn op poort 8000. Dit maakt de runtime reproduceerbaar, want wat lokaal gebouwd wordt, is in principe hetzelfde als wat in CI en Kubernetes draait.
- [requirements.txt](#): Dit legt de exacte Python-packages vast die nodig zijn. FastAPI en Uvicorn zijn voor de webservice, mysql-connector-python voor MariaDB, prometheus-client voor metrics, en python-dotenv voor het eventueel inladen van een lokaal .env-bestand. Dit heeft als voordeel dat vaste dependencyversies de builds stabiel en voorspelbaarder maken.

3.5. ./Webstack/db

Deze map bevat een init.sql script. Dit script zorgt ervoor dat bij de opstart van de DB (MariaDB) de starttabellen en data aangemaakt en ingevuld worden. In dit geval is dat de tabel 'users' en de data 'Lauwers Bjorn'

3.6. ./Webstack/frontend

De frontend is de presentatielaag van de applicatie: een statische pagina die via JavaScript met de API praat. De frontend toont de huidige gebruikersnaam, toont van welke API-container het antwoord komt en laat de naam updaten. Ze is bewust licht gehouden: HTML + CSS + een klein script, geserveerd door Lighttpd.

- index.html: Dit is de volledige webpagina plus JavaScript-logica. De pagina toont de huidige naam en API-container, haalt data op via de API en stuurt updates door als je een nieuwe naam ingeeft.
- style.css: Dit bepaalt de look-and-feel van de pagina (layout, centrering, kleuren, basisstijl).
- lighttpd.conf: De configuratie van de webserver die de frontend serveert op poort 80 met de juiste document-root en mime types.
- Dockerfile: De bouw instructies voor de frontend-image; kopieert HTML/CSS en de Lighttpd-config in de container.

3.7. ./Webstack/k8s/argocd

Deze map bevat de ArgoCD-specifieke Kubernetes-manifests voor jullie GitOps-laag. De bestanden in deze map installeren argocd niet zelf. Hiervoor werd een Helm chart gebruikt.

- argocd-ingress.yaml publiceert de ArgoCD server via NGINX Ingress op argocd.lauwersb.duckdns.org. Het binnenkomende – en uitgaande verkeer gaat over poort 443, SSL passthrough en backend HTTPS staan expliciet aan en HTTPS wordt afgedwongen
- webstack-application.yaml definieert de ArgoCD Application webstack, Het wijst naar de Git-repo en branch main. Het laat ArgoCD recursief alles onder Webstack/k8s volgen. De doelcluster is de eigen cluster, doelnamespace is project-webstack. Automated sync staat aan met prune en selfHeal.

3.8. ./Webstack/k8s/base

Deze map is de kern van de applicatie-deployment op Kubernetes: hier staat de gewenste runtime-state van de webstack zelf (DB, API, frontend, netwerktoegang en basisconfig).

- project-secrets.yaml bevat database-secrets (root password, user, user password) als Kubernetes Secret. Voordeel: gevoelige waarden worden niet gehardcodeerd in deployments, maar worden centraal en veiliger geïnjecteerd.
- db-pvc.yaml definieert een PersistentVolumeClaim van 1Gi voor MariaDB. Voordeel: databestanden blijven bewaard bij pod-herstart of rescheduling. Er is dus geen dataverlies bij gewone rollout-events.
- mariadb-congifmap.yaml ConfigMap met init.sql om database en tabel te initialiseren en startdata te laden. Voordeel: database-bootstrap worden declaratief in Git gehouden, zonder handmatige SQL-stappen.
- Praktische noot: De bestandsnaam bevat een typefout in de naam (congifmap), maar de inhoud en resource zelf werken correct.
- mariadb.yaml bevat zowel de MariaDB Service als Deployment. Voordeel: service geeft stabiele DNS-naam voor clients (mariadb-service), deployment start de DB-container met secrets, PVC-mount en init-script mount.
- api.yaml bevat API Deployment + Service. Voordeel: dit draait meerdere API-replica's, heeft health checks voor betrouwbare rolling updates, en krijgt DB-config via env/secrets. De service maakt de API intern bereikbaar voor ingress en andere workloads.
- frontend.yaml bevat frontend Deployment + Service. Voordeel: zo draaien meerdere frontend-replica's en worden ze geëxposeerd via service. De NodePort is handig als tijdelijke testroute zonder ingress.
- ingress.yaml bevat twee ingresses op dezelfde host: één voor frontend op / en één voor API op /api met rewrite. Voordeel: de browser spreekt één domein aan, terwijl ingress intern het verkeer splitst naar frontend-service en api-service; TLS en HTTPS-redirect worden afgedwongen.

3.9. ./Webstack/k8s/cert-manager

In deze map zit een script: apply-lego-tls-secrets.sh. Dit script neemt een bestaand certificaat en private key uit de lego-outputmap en maakt of updatet daaruit Kubernetes TLS secrets in meerdere namespaces.

3.10. ./Webstack/k8s/monitoring

De map monitoring bevat de eigen monitoring-aanvullingen bovenop kube-prometheus-stack.

De Helm chart installeert de monitoring stack dus zelf en daarna zegt deze map wat er specifiek gemonitord moet worden en wat er getoond en extern bereikbaar gemaakt moet worden.

- Het monitoring.yaml bestand definieert een ServiceMonitor voor de API-service. Het instrueert de Prometheus Operator om de service met het label app: api-service te scrapen op het /metrics endpoint via de http-poort. Waarom: zonder ServiceMonitor herkent Prometheus het custom API-endpoint niet automatisch. Dit sluit aan op de metrics die worden exposed in main.py en op de service-definitie in api.yaml.
- grafana-dashboard-bl.yaml is een ConfigMap die een Grafana-dashboard in JSON-formaat bevat. Het label grafana_dashboard: "1" zorgt ervoor dat de Grafana-sidecar dit dashboard automatisch importeert. Het dashboard visualiseert onder andere db_connection_latency_seconds en api_requests_total.
Waarom: Dankzij dashboard-as-code in Git is het dashboard volledig reproduceerbaar en versiebeheerbaar.
- grafana-ingress.yaml publiceert Grafana extern via een Ingress op de host grafana.lauwersb.duckdns.org,
Het routeert verkeer vanaf / naar de service prometheus-grafana, forceert HTTPS en maakt gebruik van het TLS-secret monitoring-tls. Waarom: Dit biedt veilige en gebruiksvriendelijke toegang tot Grafana van buiten de cluster.

4. Opzetten Kubernetes cluster

Na het klaarmaken van de host-pc, opstarten van de VM en installeren van de tools, kan de cluster aangemaakt worden. Dit wordt gedaan aan de hand van de configuratiebestanden die hierboven beschreven werden.

4.1. Aanmaken van de cluster, ingress en namespaces

- kind create cluster --config kind-config.yaml → maak een cluster aan op basis van het configuratiebestand.
- Installeer ingress: kubectl apply -f <https://raw.githubusercontent.com/Kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml> Hiervoor wordt een curlcommand gebruikt om een ingress-nginx manifests voor kind te installeren
- Maak de Namespaces aan: kubectl apply -f project-namespaces.yaml

Verifiëren:

- kubectl cluster-info
- kubectl get nodes -o wide
- kubectl get ingress
- kubectl get namespaces

4.2. Integreer de app in de cluster

- `kubectl apply -f k8s/base/project-secrets.yaml`
- `kubectl apply -f k8s/base/db-pvc.yaml`
- `kubectl apply -f k8s/base/mariadb-configmap.yaml`
- `kubectl apply -f k8s/base/mariadb.yaml`
- wacht tot db is opgestart voor apply van api ---
- `kubectl apply -f k8s/base/api.yaml`
- `kubectl apply -f k8s/base/frontend.yaml`
- `kubectl apply -f k8s/base/ingress.yaml`

Verifiëren:

- `kubectl get pods -n project-webstack`
- `kubectl get svc -n project-webstack`
- `kubectl get ingress -n project-webstack`

4.3. Monitoring opzetten

Prometheus + Grafana installeren met een Helm chart:

- `helm repo add prometheus-community https://prometheus-community.github.io/Helm-charts`
- `helm repo update`
- `helm install prometheus prometheus-community/kube-prometheus-stack -n monitoring --create-namespace`

Eigen configuratie toevoegen aan de monitoring:

- `kubectl apply -f k8s/monitoring/monitoring.yaml`
- `kubectl apply -f k8s/monitoring/grafana-dashboard-bl.yaml`
- `kubectl apply -f k8s/monitoring/grafana-ingress.yaml`

Verifiëren:

- `kubectl get pods -n monitoring`
- `kubectl get ingress -n monitoring`

4.4. ArgoCD opzetten:

ArgoCD installeren met Helm chart:

- `helm repo add argo https://argoproj.github.io/argo-Helm`
- `helm repo update`
- `helm install argocd argo/argo-cd --namespace argocd kubectl apply -f k8s/argocd/webstack-application.yaml`

Eigen configuratie toevoegen aan argocd

- `kubectl apply -f k8s/argocd/argocd-ingress.yaml`
- `kubectl apply -f k8s/argocd/webstack-application.yaml`

Initieel ArgoCD admin wachtwoord opvragen:

- `kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" | base64 -d; echo`

Verifiëren:

- `kubectl get pods -n argocd`
- `kubectl get ingress -n argocd`

4.5. Certificaat toevoegen en tls opzetten

Controleer certificaat:

- `openssl x509 -in /vagrant/lego/certificates/_.lauwersb.duckdns.org.crt -noout -dates -ext subjectAltName`

TLS secrets toepassen:

- `bash /vagrant/Webstack/k8s/cert-manager/apply-lego-tls-secrets.sh`

Verifiëren:

- `kubectl get secret webstack-tls -n project-webstack`
- `kubectl get secret monitoring-tls -n monitoring`
- `kubectl get secret argocd-tls -n argocd`

Voeg op de machine waarmee je test deze regels toe aan hosts-file:

- `192.168.56.10 web.lauwersb.duckdns.org`
- `192.168.56.10 grafana.lauwersb.duckdns.org`
- `192.168.56.10 argocd.lauwersb.duckdns.org`

4.6. Eindcontrole

`kubectl get pods -A`

`kubectl get ingress -A`

`curl -I https://web.lauwersb.duckdns.org`

`curl -I https://grafana.lauwersb.duckdns.org`

`curl -I https://argocd.lauwersb.duckdns.org`

4.7. Troubleshooting

4.7.1. Ingress geeft 404

Controleer host in ingress manifests:

- ``web.lauwersb.duckdns.org`` moet exact kloppen

Controle:

- `kubectl describe ingress -n project-webstack`

4.7.2. API kan DB niet bereiken

Controleer:

- `kubectl logs -n project-webstack deploy/api-deployment`
- `kubectl logs -n project-webstack deploy/mariadb-deployment`
- `kubectl get secret mariadb-secrets -n project-webstack -o yaml`

Waarom:

- Logs tonen connectiefouten
- Secret bevestigt of credentials beschikbaar zijn

4.7.3. ArgoCD overschrijft manuele Kubectl edits

Verklaring:

- ArgoCD `selfHeal` staat aan; handmatige wijzigingen worden teruggezet naar Git-state

Oplossing:

- Wijzig manifests in Git en push naar `main`



VOLG ONS

www.thomasmore.be
fb.com/ThomasMoreBE
[#WeAreMore](https://twitter.com/ThomasMoreBE)

**THOMAS
MORE**